



Fundamentos de Bases de Datos

Práctica 12: Disparadores (Triggers)

Semestre 2026-1

Profesora: Dra. Amparo López Gaona

`alg@ciencias.unam.mx`

Laboratorio: Carlos Augusto Escalona Navarro

`caen@ciencias.unam.mx`

24 de noviembre de 2025

Objetivo General

Comprender el uso de **disparadores (triggers)** en PostgreSQL, su propósito dentro de la integridad y automatización de una base de datos, y practicar su implementación en el esquema de aeropuertos.

Introducción a los disparadores

Un **disparador (trigger)** es un objeto de la base de datos que ejecuta automáticamente una función cuando ocurre un evento específico sobre una tabla.

¿Para qué sirven los disparadores?

Los triggers permiten:

- Automatizar tareas dentro de una base de datos.
- Mantener integridad adicional que no puede definirse con una restricción.
- Generar bitácoras o auditorías.
- Validar datos antes de insertar o modificar.
- Sincronizar datos entre tablas.

¿Cómo funcionan?

Un disparador está compuesto por dos partes:

1. Una **función trigger** escrita en PL/pgSQL.
2. La instrucción **CREATE TRIGGER** que indica cuándo y sobre qué tabla se ejecuta.

Eventos que pueden activar un disparador

Los triggers pueden ejecutarse ante las siguientes operaciones:

- **INSERT**
- **UPDATE**
- **DELETE**
- **TRUNCATE** (sólo AFTER)

Momento de ejecución

Los disparadores pueden ejecutarse:

- **BEFORE** antes de aplicar el cambio.
- **AFTER** después de aplicar el cambio.

Nivel de ejecución

Un trigger puede ejecutarse:

- **FOR EACH ROW** se ejecuta por cada fila afectada.
- **FOR EACH STATEMENT** se ejecuta una sola vez por sentencia.

Ejemplo:

- Insertar 10 filas con un trigger **row-level** 10 ejecuciones.
- Insertar 10 filas con un trigger **statement-level** 1 ejecución.

Ejemplos de disparadores

A continuación presentamos algunos ejemplos sencillos que utilizaremos en clase.

1. Prevenir sobreventa de boletos

```
CREATE OR REPLACE FUNCTION fn_prevenir_overbooking()  
RETURNS TRIGGER AS $$  
DECLARE  
    capacidad_avion INT;  
    boletos_vendidos INT;  
BEGIN  
    SELECT capacidad INTO capacidad_avion  
    FROM avion a  
    JOIN vuelo v ON v.id_avion = a.id_avion  
    WHERE v.id_vuelo = NEW.id_vuelo;  
  
    SELECT COUNT(*) INTO boletos_vendidos  
    FROM boleto
```

```
WHERE id_vuelo = NEW.id_vuelo;

IF boletos_vendidos >= capacidad_avion THEN
    RAISE EXCEPTION 'No se pueden vender más boletos: vuelo % lleno
    ↪ ', NEW.id_vuelo;
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_prevenir_overbooking
BEFORE INSERT ON boleto
FOR EACH ROW
EXECUTE FUNCTION fn_prevenir_overbooking();
```

2. Auditoría de cambios en estado de vuelos

```
CREATE TABLE auditoria_vuelos (
    id_log SERIAL PRIMARY KEY,
    id_vuelo INT,
    estado_anterior TEXT,
    estado_nuevo TEXT,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE OR REPLACE FUNCTION fn_auditar_vuelos()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.estado <> OLD.estado THEN
        INSERT INTO auditoria_vuelos(id_vuelo, estado_anterior,
        ↪ estado_nuevo)
        VALUES (OLD.id_vuelo, OLD.estado, NEW.estado);
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_auditar_vuelos
AFTER UPDATE ON vuelo
FOR EACH ROW
EXECUTE FUNCTION fn_auditar_vuelos();
```

3. Validación de vigencia de licencia de piloto

```
CREATE OR REPLACE FUNCTION fn_validar_licencia()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.fecha_vigencia < CURRENT_DATE THEN
        RAISE EXCEPTION 'La licencia del piloto % esta vencida', NEW.
        ↪ id_piloto;
    END IF;
END;
```

```
        RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_validar_licencia
BEFORE INSERT OR UPDATE ON piloto
FOR EACH ROW
EXECUTE FUNCTION fn_validar_licencia();
```

Actividad

El alumno deberá crear un archivo:

04_disparadores_aeropuerto.sql

que deberá incluir:

1. Tres funciones trigger en PL/pgSQL.
2. Tres triggers asociados a las funciones.
3. Comentarios explicando qué hace cada trigger.
4. Casos de prueba (INSERT/UPDATE/DELETE) para validar su funcionamiento.

Entregables

Para esta práctica, y con el fin de permitir la correcta automatización en la carga y evaluación de los scripts SQL en PostgreSQL, el equipo deberá entregar un archivo comprimido con la siguiente estructura estandarizada:

NombreEquipo_Practica12_.zip

README.txt

DOC/

 reporte_disparadores.pdf

 reporte_normalizacion.pdf

 diagrama_ER.pdf

 diagrama_relacional.pdf

 reporte_consultas_actualizado.pdf

 (documentos solicitados en prácticas anteriores)

SQL/

 DDL/

 01_DDL.sql

 02_DDL_normalizacion.sql

 (ajustes previos si los hubiera)

 DML/

 01_DML_inserts.sql

 02_funciones_aeropuerto.sql

 03_procedimientos_aeropuerto.sql

```
04_disparadores_aeropuerto.sql
05_DML_consultas.sql
06_diccionario_datos.sql
```

Reglas importantes:

- Todos los scripts en las carpetas DDL y DML deben estar **enumerados** al inicio del nombre para indicar el orden de ejecución.
- Los documentos generados en prácticas anteriores deben mantenerse y actualizarse según corresponda.
- El `README.txt` debe incluir:
 - Nombre del equipo e integrantes.
 - Lista de archivos incluidos.
 - Orden de ejecución (coincidiendo con los números de los scripts).

Evaluación

- Correcta implementación de 3 funciones y 3 triggers.
- Uso adecuado de BEFORE / AFTER y ROW / STATEMENT.
- Comentarios claros dentro del SQL.
- Funcionamiento correcto de los casos de prueba.

Dudas

En caso de dudas, contactar:

- `caen@ciencias.unam.mx`

Asunto sugerido:

[FBD-2026-1] [Duda_Practica12] - Tema de la duda